Linux 101

Design, hærdning og overvågning af en Linux-server

Linux Basics i en IT-sikkerhedskontekst

Arbejdsform: Individuel eller par

Indholdsfortegnelse

Indholdsfortegnelse 1
1. Formål med opgaven 2
2. Linux i en IT-sikkerhedskontekst — hvorfor det er relevant 3
3. Læringsmål 3
Hvor bruges dette i virkeligheden? 4
4. Opgavens moduler 4
Modul 1: VM-opsætning og netværk 5
Opgaver 5
Afleveringskrav for modulet 5
Modul 2: Filsystemet og adgangskontrol 5
Hvorfor det er relevant i en sikkerhedskontekst 5
Opgaver 5
Dokumentationskrav for modulet 5
Modul 3: Bruger- og gruppestyring 5
Hvorfor det er relevant i en sikkerhedskontekst 5
Opgaver 6
Dokumentationskrav for modulet 6
Modul 4: Firewall og netværkssikkerhed 6
Hvorfor det er relevant i en sikkerhedskontekst 6
Opgaver 6
Dokumentationskrav for modulet 6
Modul 5: System monitoring og logging 6
Hvorfor det er relevant i en sikkerhedskontekst 7
Opgaver 7
Dokumentationskrav for modulet 7
Modul 6: Shell og Bash scripting 7
Hvorfor det er relevant i en sikkerhedskontekst 7
Opgaver 7
Dokumentationskrav for modulet 7

1. Formål med opgaven

Denne opgave samler de centrale emner fra Linux Basics-forløbet — VM-opsætning, filsystem, brugere og grupper, firewall, systemovervågning, shells samt shell- og bash-scripting — i ét sammenhængende, praksisnært projekt. I stedet for at behandle emnerne isoleret, skal du designe, opsætte, sikre, dokumentere og overvåge en fuld Linux-serverkonfiguration, som en systemadministrator eller sikkerhedsmedarbejder reelt ville skulle levere i en virksomhed.

Opgaven er bygget op, så hvert modul både træner en teknisk færdighed og en sikkerhedsmæssig tankegang: du skal kunne begrunde hvorfor en konfiguration er sikker — ikke kun hvordan den laves.

Overordnet scenarie

- Du er nyansat som junior systemadministrator/sikkerhedsmedarbejder i en mindre virksomhed.
- Din opgave er at levere en produktionsklar, dokumenteret og hærdet Linux-server, som skal kunne bestå et internt sikkerhedstjek.
- Serveren skal senere kunne driftes og overvåges af andre — derfor skal alt være scriptet og dokumenteret, ikke kun klikket sammen manuelt.

2. Linux i en IT-sikkerhedskontekst — hvorfor det er relevant

Linux er ikke bare 'endnu et styresystem' i en sikkerhedsuddannelse — det er selve fundamentet for store dele af moderne IT-sikkerhedsarbejde. Her får du nogle centrale årsager til at Linux fylder så meget i IT-sikkerhedsbranchen:

- Infrastruktur: Langt størstedelen af verdens servere, cloud-miljøer og containerplatforme (fx Kubernetes, Docker) kører Linux. At kunne sikre Linux er derfor at kunne sikre selve rygraden i moderne IT-drift.
- Sikkerhedsværktøjer er Linux-native: Distroer som Kali Linux og Parrot OS, samt værktøjer som Nmap, Wireshark, Snort, Suricata og Metasploit, er bygget til eller mest udbredt på Linux.
- Firewalls og netværksudstyr: Mange firewalls, IDS/IPS-systemer og routere (fx pfSense, OPNsense) kører på en Linux- eller Unix-baseret kerne.
- Log- og overvågningsplatforme: SIEM- og logging-stacks (fx ELK/Elastic Stack, Wazuh) driftes typisk på Linux-servere i et Security Operations Center (SOC).
- Least privilege er indbygget i modellen: Linux' bruger-, gruppe- og filrettighedssystem er et konkret, håndgribeligt eksempel på et af IT-sikkerhedens vigtigste principper — mindste rettigheder (principle of least privilege).
- Åbenhed og gennemsigtighed: Open source-koden bag Linux gør det muligt at auditere, hvad systemet reelt gør — noget der er centralt for tillid i sikkerhedskritiske systemer.
- Automatisering og hastighed: I incident response tæller minutter. Kommandolinjen og shell scripting gør det muligt hurtigt at undersøge, isolere og reagere på en hændelse — langt hurtigere end via en grafisk brugerflade.

Denne opgave er derfor ikke kun en teknisk øvelse — den er en introduktion til den arbejdsmåde, du vil møde som sikkerhedsmedarbejder, systemadministrator eller pentester.

3. Læringsmål

Opgavens moduler vil sikre dig erfaring indenfor det at:

- Opsætte og netværkskonfigurere en Linux-VM sikkert fra bunden, herunder nøglebaseret SSH-adgang.
- Redegøre for Linux' filsystemhierarki (FHS) og anvende korrekte permissions og ACL'er ud fra et need-to-know-princip.
- Designe og implementere en bruger- og gruppestruktur, der afspejler roller og adskiller rettigheder (segregation of duties).
- Konfigurere en firewall efter minimal-angrebsflade-princippet og begrunde hver åben port.
- Opsætte grundlæggende systemovervågning og logging, der kan understøtte detektion af unormal aktivitet.
- Anvende shell-kommandoer effektivt til daglig drift, fejlfinding og undersøgelse.
- Skrive robuste, idempotente bash-scripts, der automatiserer opsætning, kontrol og rapportering.

Hvor bruges dette i virkeligheden?

De færdigheder, du træner i denne opgave, går igen direkte i flere IT-sikkerhedsroller:

- SOC-analytiker: Bruger dagligt Linux-baserede logging- og SIEM-platforme til at opdage og undersøge hændelser.
- Systemadministrator/DevOps: Hærder og automatiserer serveropsætninger med præcis samme principper som i denne opgave — blot i større skala.
- Penetrationstester/etisk hacker: Anvender Linux som primært arbejdsredskab og udnytter ofte netop de fejlkonfigurationer, du her lærer at undgå (åbne porte, for brede rettigheder, svage adgangskontroller).
- DevSecOps-medarbejder: Bygger 'sikkerhed ind fra starten' via Infrastructure-as-Code og automatiserede, idempotente scripts — samme tankegang som modul 6.

Målet med Linux opgavem SecureBase er derfor ikke kun at checke delmålene af, men at give dig en arbejdsmetode, du kan tage direkte med ind i en praktik- eller jobfunktion inden for IT-drift eller IT-sikkerhed.

4. Opgavens moduler

Opgaven er delt i seks moduler, der bygges oven på hinanden. Hvert modul indeholder et fagligt formål, en forklaring af den sikkerhedsmæssige relevans, konkrete opgaver samt afleveringskrav.

Modul 1: VM-opsætning og netværk

Formål: At etablere et rent, kontrolleret og reproducerbart udgangspunkt for serveren, samt sikre den første og mest kritiske indgang til systemet: fjernadgang.

Opgaver

1. Installer en Linux server-distribution (fx Debian eller Ubuntu Server) uden GUI i en VM.
2. Konfigurer statisk IP-adresse og et sigende hostname.
3. Opret en ikke-root administratorbruger og deaktiver direkte root-login via SSH.
4. Konfigurer SSH til udelukkende at acceptere nøglebaseret autentifikation (deaktiver password-login).
5. Dokumentér hvert valg med én til to sætninger om den sikkerhedsmæssige begrundelse.

Afleveringskrav for modulet

- Screenshot/log af en vellykket SSH-login med nøgle, og et forsøg med password der afvises.
- Kort dokumentation (README) af netværkskonfiguration og trufne sikkerhedsvalg.

Modul 2: Filsystemet og adgangskontrol

Formål: At forstå og anvende Linux' filsystemhierarki og rettighedsmodel korrekt, så data og programmer kun er tilgængelige for dem, der reelt har brug for adgang.

Hvorfor det er relevant i en sikkerhedskontekst

- Forkerte fil- og mapperettigheder er en af de hyppigste årsager til privilege escalation-angreb, hvor en angriber udnytter for åbne rettigheder til at opnå højere adgang.
- Forståelse af FHS (fx forskellen på /etc, /var, /home, /tmp) er nødvendig for at vide, hvor følsomme data, konfigurationer og logs bør ligge — og beskyttes.
- ACL'er gør det muligt at implementere finkornet adgangsstyring, når standard bruger/gruppe/andre-modellen ikke er tilstrækkelig præcis.

Opgaver

1. Redegør kort for formålet med de vigtigste mapper i FHS (/etc, /var, /home, /tmp, /var/log m.fl.).
2. Opret en delt projektmappe, hvor adgang styres via gruppe-rettigheder, ikke via 'chmod 777'.
3. Identificér og ret mindst to bevidst forkert konfigurerede rettigheder i et udleveret testmiljø (fx en world-writable konfigurationsfil).
4. Anvend ACL'er (getfacl/setfacl) til at give én bruger midlertidig, afgrænset adgang til en mappe uden at ændre den grundlæggende gruppestruktur.

Dokumentationskrav for modulet

- Output af ls -l/getfacl der dokumenterer 'før' og 'efter' rettigheder.
- Kort forklaring af, hvorfor 'chmod 777' undgås, og hvad der er brugt i stedet.

Modul 3: Bruger- og gruppestyring

Formål: At designe en rollebaseret adgangsstruktur, der afspejler organisationens reelle behov, og som understøtter sporbarhed og ansvarlighed.

Hvorfor det er relevant i en sikkerhedskontekst

- Segregation of duties (adskillelse af ansvarsområder) forhindrer, at én kompromitteret konto giver adgang til alt — et centralt princip i governance og compliance (fx ISO 27001).
- Individuelle brugerkonti (frem for delte konti) er en forudsætning for troværdig logning: du skal altid kunne se hvem der gjorde hvad.
- Korrekt brug af sudo med begrænsede rettigheder frem for konstant root-adgang reducerer skadesomfanget, hvis en konto bliver kompromitteret.

Opgaver

1. Opret mindst tre brugerroller, fx admin, developer og guest, med forskellige rettigheder.
2. Konfigurer grupper, så developer-gruppen har skriveadgang til et fælles projektområde, mens guest kun har læseadgang.
3. Konfigurer sudo-rettigheder granulært via /etc/sudoers.d/, så admin-brugeren kun kan udføre specifikke, nødvendige kommandoer som root — ikke ubegrænset sudo.
4. Dokumentér din rollestruktur i et simpelt skema (bruger → gruppe → rettigheder → begrundelse).

Dokumentationskrav for modulet

- Rolleskema (kan være en tabel i README).
- Uddrag af /etc/group, id-output for hver bruger, og relevante sudoers-regler.

Modul 4: Firewall og netværkssikkerhed

Formål: At reducere serverens angrebsflade til et minimum ved kun at tillade den netværkstrafik, der er strengt nødvendig for systemets funktion.

Hvorfor det er relevant i en sikkerhedskontekst

- Firewall-konfiguration er en direkte anvendelse af 'default deny'-princippet: alt blokeres som udgangspunkt, og kun det nødvendige åbnes eksplicit.
- De fleste Linux-baserede firewalls (ufw, nftables, iptables) er de samme teknologier, der ligger til grund for kommercielle netværkssikkerhedsprodukter — færdigheden er direkte overførbar.
- En veldokumenteret firewall-konfiguration er ofte det første, en sikkerhedsrevisor eller pentester kigger på ved en systemgennemgang.

Opgaver

1. Konfigurer ufw eller nftables, så al indgående trafik som udgangspunkt blokeres.
2. Åbn udelukkende de porte, systemet reelt har brug for (fx SSH på en ikke-standard port, evt. en simpel webservice).
3. Begræns SSH-adgang til et defineret IP-interval, hvis scenariet tillader det.
4. Dokumentér hver åben port med begrundelse: hvilken tjeneste, hvorfor den er nødvendig, og hvilken risiko den introducerer.

Dokumentationskrav for modulet

- Eksport af de aktive firewall-regler (fx ufw status verbose).
- Portbegrundelsestabel: port, protokol, tjeneste, begrundelse.

Modul 5: System monitoring og logging

Formål: At kunne opdage unormal aktivitet, ressourceproblemer og potentielle sikkerhedshændelser, før de udvikler sig til reelle problemer.

Hvorfor det er relevant i en sikkerhedskontekst

- 'Du kan ikke opdage det, du ikke logger' — logging og overvågning er forudsætningen for enhver form for detektion og incident response.
- System- og sikkerhedslogs (fx auth.log/journalctl) er ofte det første sted, en analytiker kigger efter tegn på uautoriserede loginforsøg eller misbrug.
- Automatiseret overvågning af ressourcer (CPU, disk, hukommelse) kan afsløre både driftsproblemer og ondsindet aktivitet, fx et system der bruges til crypto-mining eller et fyldt filsystem forårsaget af et angreb.

Opgaver

1. Opsæt logrotation, så logfiler ikke vokser ukontrolleret og fylder disken.
2. Skriv et overvågningsscript, der med jævne mellemrum (cron) logger CPU-, disk- og hukommelsesforbrug til en central logfil.
3. Undersøg og dokumentér, hvordan du via journalctl/auth.log kan identificere mislykkede loginforsøg.
4. Definér mindst to simple tærskelværdier (thresholds), der bør trigge en alarm/besked (fx diskforbrug over 85%).

Dokumentationskrav for modulet

- Uddrag af logfil genereret af overvågningsscriptet.
- Kort beskrivelse af, hvordan et unormalt mønster i loginforsøg ville se ud i dine logs.

Modul 6: Shell og Bash scripting

Formål: At kunne automatisere opsætning, kontrol og rapportering, så konfigurationen er reproducerbar, konsistent og ikke afhængig af manuelle, fejlbarlige trin.

Hvorfor det er relevant i en sikkerhedskontekst

- Automatisering reducerer 'menneskelige fejl-konfigurationer', som er en af de hyppigste årsager til sikkerhedshuller i produktionsmiljøer.
- Idempotente scripts (scripts der kan køres flere gange uden at ødelægge noget) er en forudsætning for pålidelig, gentagelig systemhærdning — fx i et Infrastructure-as-Code-mindset.
- I incident response er evnen til hurtigt at skrive et lille script, der undersøger et system (processer, forbindelser, filer), afgørende for at kunne reagere, mens en hændelse stadig udvikler sig.

Opgaver

1. Skriv ét eller flere modulopdelte bash-scripts (setup.sh m.fl.), der automatiserer så meget af modul 1-5 som muligt.
2. Scriptene skal være idempotente: at køre dem igen må ikke skabe fejl eller duplikerede konfigurationer.
3. Skriv et selvstændigt 'healthcheck.sh'-script, der tjekker og rapporterer: firewall-status, ledig diskplads, aktive/loggede-ind brugere, samt om der findes brugere med UID 0 ud over root.
4. Brug funktioner, variable, betingelser og fejlhåndtering (fx set -euo pipefail) korrekt i scriptene.

Dokumentationskrav for modulet

- Alle scripts, kommenteret og med et kort afsnit om, hvordan de køres.
- Eksempeloutput fra healthcheck.sh på det færdige system.